

Payout System — Developer Mapping Doc

Base: `new tech` (Airtable) **Script:** `payout_service.js` (Node, run via `npm run once`, polls Airtable) **Goal of this doc:** map the website submission flow → Airtable fields → payout calculation, and flag the gaps that need to be built.

1. System components

1. **Website form** — clipper submits a video, selects a Creator, adds a description. Views are tracked automatically afterward.
 2. **Discord/Slack bot** — currently the intake path for raw clips (not the website form directly feeding Airtable yet, per earlier discussion — confirm with dev which is authoritative).
 3. **Airtable base (`new tech`)** — 4 tables: `Campaigns`, `Team Members`, `Submissions`, `Clippers`.
 4. **`payout_service.js`** — polls `Submissions`, calculates splits, writes payout fields back.
-

2. Airtable schema (as it exists today)

Table: `Campaigns`

Field	Type	Notes
Streamer/Creator	multilineText	Primary field
Campaign Name	multilineText	
Rate Per Million (\$)	number	Used by script to compute payouts from Views
Status	singleSelect	
Notes	multilineText	
submissions	link → Submissions	

Table: `Team Members`

Field	Type	Notes
Name	singleLineText	Primary field
Discord Handle	singleLineText	
Role	singleSelect	Options: Clipper , Account Manager , Lead Manager — ⚠️ no "Editor" option exists
Manager	link → Team Members	self-referencing
Status	singleSelect	no choices configured yet
Payment Handle	singleLineText	
submissions (link)	link → Submissions	inverse of Account Manager on Submissions
Submissions 4	link → Submissions	inverse of Editor on Submissions
Submissions 5	link → Submissions	inverse of Reviewed By on Submissions
submissions 2 / submissions 3	singleLineText	⚠️ look like leftover/duplicate text fields, not linked records — likely safe to delete, confirm with dev first
Manual sort	manualSort	
From field: Manager	link	lookup-through field

Table: **Submissions** (the core table)

Field	Type	Notes
Submission ID	number	Primary field
Clipper	link → Clippers table	⚠️ NOT Team Members — see Section 4
Discord Username (from Clipper) 2	lookup	
Account Manager	link → Team Members	
Editor	link → Team Members	⚠️ see Section 4

Campaign	link → Campaigns	
Is AM's Own Clip	checkbox	
Clip Type	singleSelect	Options: Raw , Stolen , Original-Edited , Raw-Split Edit
Platform	singleSelect	
Views	number	
Clipper % / AM % / Editor % / Owner %	number	written by script
Clipper Payout (\$) / AM Payout (\$) / Editor Payout (\$) / Owner Payout (\$) / Total Payout (\$)	number	written by script
Last Calculated Views	number	script's dedupe marker — see Section 5
Rate Used (\$/mil)	number	
Review Status / Deny Reason / Flag Reason / Reviewed By	—	moderation fields
Video / Posted URL	url	
Submissions / Clippers / Clippers 2	singleLineText	⚠ leftover plain-text fields, not the linked-record versions — likely dead fields

Table: [Clippers](#)

Field	Type	Notes
Clipper ID	singleLineText	Primary field
Discord ID	singleLineText	Looks like the match key the bot uses to auto-identify who submitted
Discord Username	singleLineText	
Submissions	link → Submissions	inverse of Clipper

3. Website form → Airtable mapping

Website field	Maps to	Table
Video (link/upload)	Video	Submissions
Creator selected	Campaign (linked record)	Submissions → Campaigns
Description	<i>(no current field — confirm if this should be Flag Reason, Notes, or a new field)</i>	—
Views (auto-tracked)	Views	Submissions
Submitter identity	Clipper (linked record, matched via Discord ID → Clippers table)	Submissions → Clippers

Open question for dev: does the website form write directly to Airtable (via API), or does it go through the Discord/Slack bot first? This determines whether "Clipper" gets linked at form-submit time or at a later bot-processing step.

4. ⚠️ Key gap: Clipper and Editor point to two different tables

- **Clipper** → links to the **Clippers** table (lightweight: just Discord ID/Username, auto-creatable, no onboarding needed).
- **Editor** (and Account Manager, Reviewed By) → links to the **Team Members** table, which has a **Role** field with only **Clipper / Account Manager / Lead Manager** — no **Editor** role exists yet.

This means: **whoever edits a raw clip must already exist as a Team Members record** — but the intake system (Discord/Slack bot, "1 of 100 possible editors") is built around the lightweight **Clippers** table, not **Team Members**.

Decision needed before building the editor-claim flow:

- **Option A** — Add **Editor** to the **Role** picklist in Team Members, and require editors to be onboarded as Team Members (more overhead, but keeps payment/role data centralized).
- **Option B** — Point the **Editor** field on Submissions at the **Clippers** table instead (same lightweight model as Clipper), so any of the 100 people can be linked without separate onboarding.

Recommend **Option B** given the "could be anyone in 100 people" scale — but it's a real schema change (repointing a linked-record field's target table), so flag it for the dev to confirm before touching it.

5. Payout calculation logic (from `payout_service.js`)

Script polls Submissions, skips a row if:

```
if (!views || views === lastCalculatedViews) { skipped++; continue; }
```

- i.e., skips if Views is blank, or unchanged since the last successful calculation.
 - Split percentages depend on **Clip Type** + **Platform**:
 - **Raw-Split Edit**: Editor gets a real cut (35% TikTok/X/IG, 40% YouTube) — the only clip type where Editor % is non-zero. Requires both **Clipper** and **Editor** linked, or the row fails.
 - **Original-Edited**: same person sources + edits; Editor % is 0 by design, the editing work is folded into a higher Clipper % (55%/60%).
 - Other types (**Raw**, **Stolen**, AM's own clip) follow their own fixed splits.
 - **Caveat**: if the split logic changes, already-calculated rows won't auto-recalculate unless **Last Calculated Views** is cleared — there's no version marker to force a backfill. Worth adding one if the split rules are still evolving.
-

6. To-do list for the developer

1. **Decide & implement Clipper vs. Editor table targeting** (Section 4) — this blocks the "editor claims an edit" feature.
2. **Build the editor-claim step**: a bot command/reaction/thread-reply that identifies the editor (via Discord ID, same pattern as Clipper matching) and PATCHes the Submission row: link Editor + set **Clip Type** = **Raw-Split Edit**.
3. **Clarify the Description field mapping** from the website form — no current Airtable field matches it.
4. **Clean up duplicate fields**: **submissions 2/submissions 3** on Team Members, and **Submissions/Clippers/Clippers 2** (plain text) on Submissions — confirm these are dead before deleting.
5. **Optional robustness fix**: add a **Split Logic Version** field so future split-rule changes can force recalculation instead of requiring a manual clear of **Last Calculated Views**.